Title: Multi-Container Runtime — TODO Solutions

1. bounded_buffer_push (engine.c)

```
int bounded_buffer_push(bounded_buffer_t *buffer, const log_item_t *item)
{
    pthread_mutex_lock(&buffer->mutex);
    while (buffer->count == LOG_BUFFER_CAPACITY && !buffer->shutting_down)
        pthread_cond_wait(&buffer->not_full, &buffer->mutex);
    if (buffer->shutting_down) {
        pthread_mutex_unlock(&buffer->mutex);
        return -1;
    }
    buffer->items[buffer->tail] = *item;
    buffer->tail = (buffer->tail + 1) % LOG_BUFFER_CAPACITY;
    buffer->count++;
    pthread_cond_signal(&buffer->not_empty);
    pthread_mutex_unlock(&buffer->mutex);
    return 0;
}
```

2. bounded_buffer_pop (engine.c)

```
int bounded_buffer_pop(bounded_buffer_t *buffer, log_item_t *item)
{
    pthread_mutex_lock(&buffer->mutex);
    while (buffer->count == 0 && !buffer->shutting_down)
        pthread_cond_wait(&buffer->not_empty, &buffer->mutex);
    if (buffer->count == 0 && buffer->shutting_down) {
        pthread_mutex_unlock(&buffer->mutex);
```

```
        return -1;
    }
    *item = buffer->items[buffer->head];
    buffer->head = (buffer->head + 1) % LOG_BUFFER_CAPACITY;
    buffer->count--;
    pthread_cond_signal(&buffer->not_full);
    pthread_mutex_unlock(&buffer->mutex);
    return 0;
}
```

3. logging_thread (engine.c)

```
  void *logging_thread(void *arg)
{
    supervisor_ctx_t *ctx = (supervisor_ctx_t *)arg;
    log_item_t item;

    while (1) {
        if (bounded_buffer_pop(&ctx->log_buffer, &item) != 0)
            break;

        char path[PATH_MAX];
        snprintf(path, PATH_MAX, "%s/%s.log", LOG_DIR, item.container_id);

        int fd = open(path, O_WRONLY | O_CREAT | O_APPEND, 0644);
        if (fd < 0) {
            perror("logging_thread: open");
            continue;
        }
        ssize_t written = write(fd, item.data, item.length);
        if (written < 0)
```

```
        perror("logging_thread: write");
    close(fd);
  }
  return NULL;
}
```

4. child_fn (engine.c)

```
 int child_fn(void *arg)
{
  child_config_t *cfg = (child_config_t *)arg;

  if (cfg == NULL)
    return 1;

  if (cfg->log_write_fd >= 0) {
    dup2(cfg->log_write_fd, STDOUT_FILENO);
    dup2(cfg->log_write_fd, STDERR_FILENO);
    close(cfg->log_write_fd);
  }
  if (chroot(cfg->rootfs) != 0) {
    perror("child_fn: chroot");
    return 1;
  }
  if (chdir("/") != 0) {
    perror("child_fn: chdir");
    return 1;
  }
  if (mount("proc", "/proc", "proc", 0, NULL) != 0)
    perror("child_fn: mount /proc");

  if (cfg->nice_value != 0) {
```

```
    if (setpriority(PRIO_PROCESS, 0, cfg->nice_value) != 0)
        perror("child_fn: setpriority");
  }
  char cmd_copy[CHILD_COMMAND_LEN];
  strncpy(cmd_copy, cfg->command, CHILD_COMMAND_LEN - 1);

  char *args[64];
  int nargs = 0;
  char *token = strtok(cmd_copy, " ");
  while (token && nargs < 63) {
    args[nargs++] = token;
    token = strtok(NULL, " ");
  }
  args[nargs] = NULL;

  if (nargs == 0) {
    fprintf(stderr, "child_fn: empty command\n");
    return 1;
  }

  execv(args[0], args);
  perror("child_fn: execv");
  return 1;
}
```

5. run_supervisor (engine.c)

```
static int run_supervisor(const char *rootfs)
{
  supervisor_ctx_t ctx;
  int rc;
```

```
memset(&ctx, 0, sizeof(ctx));
ctx.server_fd  = -1;
ctx.monitor_fd = -1;

/* Install signal handlers before anything else */
g_ctx = &ctx;
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sa.sa_handler = sigchld_handler;
sa.sa_flags   = SA_RESTART | SA_NOCLDSTOP;
sigaction(SIGCHLD, &sa, NULL);
sa.sa_handler = sigterm_handler;
sa.sa_flags   = SA_RESTART;
sigaction(SIGINT,  &sa, NULL);
sigaction(SIGTERM, &sa, NULL);

mkdir(LOG_DIR, 0755);

rc = pthread_mutex_init(&ctx.metadata_lock, NULL);
if (rc != 0) { errno = rc; perror("pthread_mutex_init"); return 1; }

rc = bounded_buffer_init(&ctx.log_buffer);
if (rc != 0) {
   errno = rc; perror("bounded_buffer_init");
   pthread_mutex_destroy(&ctx.metadata_lock);
   return 1;
}

ctx.monitor_fd = open("/dev/container_monitor", O_RDWR);
if (ctx.monitor_fd < 0)
   perror("open /dev/container_monitor"); /* non-fatal */
```

```
unlink(CONTROL_PATH);
ctx.server_fd = socket(AF_UNIX, SOCK_STREAM, 0);
if (ctx.server_fd < 0) { perror("socket"); goto cleanup; }

struct sockaddr_un addr;
memset(&addr, 0, sizeof(addr));
addr.sun_family = AF_UNIX;
strncpy(addr.sun_path, CONTROL_PATH, sizeof(addr.sun_path) - 1);

if (bind(ctx.server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
   perror("bind"); goto cleanup;
}
if (listen(ctx.server_fd, 8) < 0) {
   perror("listen"); goto cleanup;
}

rc = pthread_create(&ctx.logger_thread, NULL, logging_thread, &ctx);
if (rc != 0) { errno = rc; perror("pthread_create"); goto cleanup; }

fprintf(stderr, "[supervisor] started, rootfs=%s, socket=%s\n",
       rootfs, CONTROL_PATH);

while (!ctx.should_stop) {
   int client_fd = accept(ctx.server_fd, NULL, NULL);
   if (client_fd < 0) {
       if (errno == EINTR) continue;
       perror("accept");
       break;
   }
```

```
control_request_t req;
control_response_t resp;
memset(&resp, 0, sizeof(resp));

ssize_t n = read(client_fd, &req, sizeof(req));
if (n != (ssize_t)sizeof(req)) {
    close(client_fd);
    continue;
}

switch (req.kind) {

/* ----- CMD_START: launch and return immediately ----- */
case CMD_START: {
    pthread_mutex_lock(&ctx.metadata_lock);
    int dup = (container_list_find(&ctx, req.container_id) != NULL);
    pthread_mutex_unlock(&ctx.metadata_lock);

    if (dup) {
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message),
                 "Container '%s' already exists", req.container_id);
        break;
    }

    int pipefd[2];
    pid_t pid = spawn_container(&ctx, &req, pipefd);
    if (pid < 0) {
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message), "Failed to start container");
        break;
```

```
    }

    resp.status = 0;
    snprintf(resp.message, sizeof(resp.message),
            "Container '%s' started, pid=%d", req.container_id, pid);
    break;
}

/* ----- CMD_RUN: launch and block until exit ----- */
case CMD_RUN: {
    pthread_mutex_lock(&ctx.metadata_lock);
    int dup = (container_list_find(&ctx, req.container_id) != NULL);
    pthread_mutex_unlock(&ctx.metadata_lock);

    if (dup) {
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message),
                "Container '%s' already exists", req.container_id);
        ssize_t w = write(client_fd, &resp, sizeof(resp));
        if (w < 0) perror("write response");
        close(client_fd);
        continue;
    }

    int pipefd[2];
    pid_t pid = spawn_container(&ctx, &req, pipefd);
    if (pid < 0) {
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message), "Failed to start container");
        ssize_t w = write(client_fd, &resp, sizeof(resp));
        if (w < 0) perror("write response");
```

```
    close(client_fd);
    continue;
}

/* Block until container exits */
int wstatus;
pid_t exited = waitpid(pid, &wstatus, 0);

pthread_mutex_lock(&ctx.metadata_lock);
container_record_t *r = container_list_find(&ctx, req.container_id);
if (r) {
    if (WIFEXITED(wstatus)) {
        r->state     = CONTAINER_EXITED;
        r->exit_code = WEXITSTATUS(wstatus);
    } else if (WIFSIGNALED(wstatus)) {
        r->state       = CONTAINER_KILLED;
        r->exit_signal = WTERMSIG(wstatus);
    }
}
pthread_mutex_unlock(&ctx.metadata_lock);

if (ctx.monitor_fd >= 0)
    unregister_from_monitor(ctx.monitor_fd, req.container_id, pid);

resp.status = (exited > 0 && WIFEXITED(wstatus))
            ? WEXITSTATUS(wstatus) : -1;
snprintf(resp.message, sizeof(resp.message),
         "Container '%s' exited with code %d",
         req.container_id, resp.status);

ssize_t w = write(client_fd, &resp, sizeof(resp));
```

```
    if (w < 0) perror("write response");
    close(client_fd);
    continue;
}

/* ----- CMD_PS ----- */
case CMD_PS: {
    pthread_mutex_lock(&ctx.metadata_lock);
    container_record_t *cur = ctx.containers;
    char *p = resp.message;
    int rem = (int)sizeof(resp.message);
    while (cur && rem > 1) {
        int written = snprintf(p, rem, "%s %s pid=%d\n",
                               cur->id,
                               state_to_string(cur->state),
                               cur->host_pid);
        p   += written;
        rem -= written;
        cur  = cur->next;
    }
    pthread_mutex_unlock(&ctx.metadata_lock);
    resp.status = 0;
    break;
}

/* ----- CMD_STOP ----- */
case CMD_STOP: {
    pthread_mutex_lock(&ctx.metadata_lock);
    container_record_t *rec = container_list_find(&ctx, req.container_id);
    pthread_mutex_unlock(&ctx.metadata_lock);
```

```
    if (!rec) {
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message),
                 "Container '%s' not found", req.container_id);
        break;
    }

    kill(rec->host_pid, SIGTERM);
    rec->state = CONTAINER_STOPPED;

    if (ctx.monitor_fd >= 0)
        unregister_from_monitor(ctx.monitor_fd,
                                req.container_id, rec->host_pid);

    resp.status = 0;
    snprintf(resp.message, sizeof(resp.message),
             "Container '%s' stopped", req.container_id);
    break;
}

/* ----- CMD_LOGS ----- */
case CMD_LOGS: {
    pthread_mutex_lock(&ctx.metadata_lock);
    container_record_t *rec = container_list_find(&ctx, req.container_id);
    pthread_mutex_unlock(&ctx.metadata_lock);

    if (!rec) {
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message),
                 "Container '%s' not found", req.container_id);
        break;
```

```
        }
        resp.status = 0;
        snprintf(resp.message, sizeof(resp.message),
                "Log file: %s", rec->log_path);
        break;
      }

      default:
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message), "Unknown command");
        break;
      }

      ssize_t w = write(client_fd, &resp, sizeof(resp));
      if (w < 0) perror("write response");
      close(client_fd);
    }

cleanup:
    if (ctx.server_fd  >= 0) close(ctx.server_fd);
    if (ctx.monitor_fd >= 0) close(ctx.monitor_fd);
    unlink(CONTROL_PATH);

    bounded_buffer_begin_shutdown(&ctx.log_buffer);
    pthread_join(ctx.logger_thread, NULL);
    bounded_buffer_destroy(&ctx.log_buffer);
    pthread_mutex_destroy(&ctx.metadata_lock);
    return 0;
}
```

6. send_control_request (engine.c)

```
static int send_control_request(const control_request_t *req)
{
    int fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (fd < 0) { perror("socket"); return 1; }

    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, CONTROL_PATH, sizeof(addr.sun_path) - 1);

    if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("connect: is the supervisor running?");
        close(fd);
        return 1;
    }

    ssize_t w = write(fd, req, sizeof(*req));
    if (w != (ssize_t)sizeof(*req)) {
        perror("write request");
        close(fd);
        return 1;
    }

    control_response_t resp;
    ssize_t n = read(fd, &resp, sizeof(resp));
    close(fd);

    if (n != (ssize_t)sizeof(resp)) {
        fprintf(stderr, "Short response from supervisor\n");
        return 1;
    }
```

```
    printf("%s\n", resp.message);
    return resp.status == 0 ? 0 : 1;
}
```

7. monitor.c — TODO 1: struct monitored_entry

```
  struct monitored_entry {
    pid_t pid;
    char container_id[64];
    unsigned long soft_limit_bytes;
    unsigned long hard_limit_bytes;
    int soft_warned;        /* 1 after soft warning has been emitted */
    struct list_head list;
};
```

8. monitor.c — TODO 2: global list and lock

```
static LIST_HEAD(monitored_list);
static DEFINE_MUTEX(monitored_lock);
```

9. monitor.c — TODO 3: timer_callback

```
static void timer_callback(struct timer_list *t)
{
    struct monitored_entry *entry, *tmp;

    mutex_lock(&monitored_lock);

    list_for_each_entry_safe(entry, tmp, &monitored_list, list) {
        long rss = get_rss_bytes(entry->pid);

        /* Process has exited — remove stale entry */
```

```
        if (rss < 0) {
            printk(KERN_INFO
                   "[container_monitor] Process exited container=%s pid=%d, removing\n",
                   entry->container_id, entry->pid);
            list_del(&entry->list);
            kfree(entry);
            continue;
        }

        /* Hard limit check — kill and remove */
        if ((unsigned long)rss > entry->hard_limit_bytes) {
            kill_process(entry->container_id, entry->pid,
                         entry->hard_limit_bytes, rss);
            list_del(&entry->list);
            kfree(entry);
            continue;
        }

        /* Soft limit check — warn once */
        if ((unsigned long)rss > entry->soft_limit_bytes && !entry->soft_warned) {
            log_soft_limit_event(entry->container_id, entry->pid,
                                 entry->soft_limit_bytes, rss);
            entry->soft_warned = 1;
        }
    }

    mutex_unlock(&monitored_lock);

    mod_timer(&monitor_timer, jiffies + CHECK_INTERVAL_SEC * HZ);
}
```

10. monitor.c — TODO 4: MONITOR_REGISTER

```
static long monitor_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
    struct monitor_request req;

    (void)f;

    if (cmd != MONITOR_REGISTER && cmd != MONITOR_UNREGISTER)
        return -EINVAL;

    if (copy_from_user(&req, (struct monitor_request __user *)arg, sizeof(req)))
        return -EFAULT;

    if (cmd == MONITOR_REGISTER) {
        struct monitored_entry *entry;
```

11. monitor.c — TODO 5: MONITOR_UNREGISTER

```
    if (cmd != MONITOR_REGISTER && cmd != MONITOR_UNREGISTER)
        return -EINVAL;
```

12. monitor.c — TODO 6: monitor_exit cleanup

```
static void __exit monitor_exit(void)
{
    timer_delete_sync(&monitor_timer);

    /* TODO 6: Free all remaining entries */
    {
        struct monitored_entry *entry, *tmp;
```

```
        mutex_lock(&monitored_lock);
        list_for_each_entry_safe(entry, tmp, &monitored_list, list) {
            list_del(&entry->list);
            kfree(entry);
        }
        mutex_unlock(&monitored_lock);
    }

    cdev_del(&c_dev);
    device_destroy(cl, dev_num);
    class_destroy(cl);
    unregister_chrdev_region(dev_num, 1);

    printk(KERN_INFO "[container_monitor] Module unloaded.\n");
}
```